

ESCUELA SUPERIOR POLITÉCNICA DE CHIMBORAZO

FACULTAD DE INFORMÁTICA Y ELECTRÓNICA

CARRERA DE TECNOLOGÍAS DE LA INFORMACIÓN

SISTEMA WEB DE MONITOREO Y GESTIÓN DE LA INFRAESTRUCTURA DE REDES

MANUAL TÉCNICO DEL PROYECTO INTEGRADOR DE TECNOLOGÍAS DE LA INFORMACIÓN

Roles de Ingeniería:

- Líder Técnico y Arquitecto Backend
- Desarrollador Frontend y Diseñador UI/UX
- Especialista en Bases de Datos y QA Lead
- Ingeniero DevOps y Redes

Docentes Responsables:

- Infraestructura TI: Ing. Tatiana Landivar
- Tecnología Web: Ing. Pedro Aguilar
- Conmutación y Enrut.: Ing. Juan Pazmiño
- Big Data: Ing. Geovanny Silva
- Diseño y Multimedia: Ing. Ramiro Bonilla

Período Académico: Quinto Semestre (PAO 5)

Manual Técnico del Proyecto Integrador

Orellana - Ecuador

Noviembre de 2026

Índice general

1. Introducción y Planteamiento del Proyecto **5**

1.1. Introducción 5

1.2. Problema de Estudio 5

1.3. Justificación 5

2. Objetivos **7**

2.1. Objetivo General 7

2.2. Objetivos Específicos 7

3. Alcance y Limitaciones **8**

3.1. Alcance 8

3.2. Limitaciones 8

4. Planificación y Metodologías de Desarrollo **9**

4.1. Metodología Seleccionada y Justificación 9

4.2. Planificación 9

5. Desarrollo de la Solución Tecnológica **11**

5.1. Análisis y Especificación de Requerimientos 11

5.1.1. Requerimientos Funcionales 11

5.1.2. Requerimientos No Funcionales 13

5.1.3. Actores del Sistema 13

5.2. Diseño de la Solución Tecnológica 14

5.2.1. Arquitectura del Sistema: Monolito Modular Dirigido por Eventos (EDA) 14

5.2.2. Diagrama de Componentes y Flujo de Eventos 15

5.2.3. Catálogo de Eventos del Bus Interno 17

5.2.4. Diagrama de Secuencia: Detección de Caída 17

5.2.5. Modelo de Datos (MongoDB) 18

5.2.6. Topología de Red 19
5.2.7. Diagrama de Despliegue 19
5.3. Implementación de la Solución 21
5.3.1. Stack Tecnológico 21
5.3.2. Organización de Módulos del Backend 21
5.3.3. Matrices de Justificación Técnica 22
5.4. Pruebas y Validación Técnica 23
5.4.1. Criterios y Condiciones de Prueba 23
5.4.2. Casos de Prueba Principales 23
5.4.3. Correspondencia Requerimientos - Validación 23

6. Operación, Mantenimiento y Soporte 25
6.1. Operación Técnica 25
6.2. Mantenimiento 25
6.3. Respaldo y Recuperación 25
6.4. Errores Frecuentes y Soluciones 26
6.5. Limitaciones Conocidas 26

7. Conclusiones y Recomendaciones 27
7.1. Conclusiones 27
7.2. Recomendaciones 27

8. Anexos y Entregables 29
8.1. Repositorio del Proyecto 29
8.2. Manual de Usuario 29
8.3. Evidencias Complementarias 29

Índice de figuras

4.1. Flujo de sprints del proyecto 9

5.1. Diagrama de casos de uso del sistema 14
5.2. Diagrama de arquitectura general del sistema 15
5.3. Diagrama de componentes y flujo de eventos del sistema (EDA) 16
5.4. Diagrama de secuencia: detección de caída de un dispositivo 17
5.5. Modelo de datos del sistema (colecciones MongoDB) 18
5.6. Topología de red del laboratorio 19
5.7. Diagrama de despliegue del sistema 20
5.8. Capas técnicas del stack tecnológico 21

Índice de tablas

4.1. Cronograma de fases y sprints del proyecto 10

5.1. Especificación completa de requerimientos funcionales 12
5.2. Especificación completa de requerimientos no funcionales 13
5.3. Catálogo de eventos del bus interno 17
5.4. Tecnologías utilizadas y su función 21
5.5. Justificación de decisiones de arquitectura y comunicación 22
5.6. Justificación de persistencia, monitoreo e infraestructura 23
5.7. Casos de prueba y resultados esperados 24
5.8. Trazabilidad de requerimientos a casos de prueba 24

6.1. Errores frecuentes y soluciones 26

Capítulo 1

Introducción y Planteamiento del Proyecto

1.1 Introducción

El presente documento constituye el manual técnico del proyecto integrador desarrollado durante el quinto semestre (PAO 5) de la Carrera de Tecnologías de la Información en la Escuela Superior Politécnica de Chimborazo (ESPOCH), sede Orellana.

El proyecto consiste en un sistema web de monitoreo y gestión de la infraestructura de redes, diseñado para operar en el laboratorio de cómputo de la universidad. La solución permite descubrir, registrar, supervisar y administrar los dispositivos conectados a una red local (LAN) mediante una interfaz web en tiempo real.

El documento se organiza siguiendo la estructura del Manual Técnico del Proyecto Integrador de Tecnologías de la Información, cubriendo desde el planteamiento del problema hasta las pruebas de validación, la operación y el mantenimiento del sistema.

1.2 Problema de Estudio

En el laboratorio de cómputo de la ESPOCH sede Orellana, la gestión de los dispositivos conectados a la red se realiza de forma manual y reactiva. No existe un sistema centralizado que permita conocer en tiempo real qué equipos están conectados, cuáles presentan problemas de conectividad o cuándo un dispositivo se ha desconectado.

Esta situación genera demoras en la detección de fallos, dificultad para mantener un inventario actualizado de los equipos de red y ausencia de registros históricos que permitan analizar patrones de disponibilidad. El personal encargado solo detecta los problemas cuando un usuario reporta una incidencia, lo que incrementa los tiempos de respuesta y reduce la eficiencia operativa del laboratorio.

1.3 Justificación

La implementación de un sistema de monitoreo automatizado aborda directamente las necesidades identificadas en el laboratorio:

- Permite la detección proactiva de desconexiones y cambios de estado en la red, reduciendo el tiempo de respuesta ante incidencias.

- Centraliza la información de todos los dispositivos en un inventario digital consultable y actualizable.
- Genera registros históricos que facilitan el análisis de patrones de disponibilidad y la toma de decisiones sobre la infraestructura.
- Integra conocimientos de cinco asignaturas del semestre: Infraestructura TI, Tecnología Web, Conmutación y Enrutamiento, Big Data y Diseño y Tecnología Multimedia.
- Utiliza tecnologías de código abierto disponibles sin costo de licenciamiento, lo que garantiza la viabilidad del proyecto dentro de las restricciones presupuestarias del entorno académico.

Capítulo 2

Objetivos

2.1 Objetivo General

Desarrollar un sistema web de monitoreo y gestión centralizada que permita descubrir, visualizar y supervisar en tiempo real los dispositivos conectados a la red del laboratorio de cómputo, facilitando la identificación de cambios de disponibilidad y la administración de la información de la red.

2.2 Objetivos Específicos

1. Analizar el problema de la gestión manual de la infraestructura de red del laboratorio, identificando los requerimientos funcionales y no funcionales del sistema.
2. Diseñar la arquitectura de software basada en eventos (EDA) que permita la comunicación en tiempo real entre los módulos de monitoreo, alertas, persistencia y notificaciones.
3. Implementar el backend con Node.js y Express, el frontend con React 18 y la base de datos con MongoDB 7.0, integrando WebSockets para la transmisión en tiempo real.
4. Implementar los servicios de descubrimiento de dispositivos mediante ARP-scan y monitoreo de disponibilidad mediante ICMP ping.
5. Validar el funcionamiento del sistema mediante pruebas de integración y pruebas de aceptación que verifiquen el cumplimiento de los requerimientos establecidos.

Capítulo 3

Alcance y Limitaciones

3.1 Alcance

El sistema contempla las siguientes funcionalidades para su primera versión:

- Monitoreo de una red LAN con aproximadamente 30 dispositivos.
- Descubrimiento automático de dispositivos mediante ARP-scan y Nmap.
- Comprobaciones periódicas de disponibilidad mediante ICMP ping.
- Registro manual y automático de dispositivos en el inventario.
- Panel web con información en tiempo real del estado de la red.
- Sistema de alertas e incidencias para cambios de disponibilidad.
- Autenticación y control de acceso basado en roles (Administrador y Operador).
- Historial de eventos y registros de auditoría.
- Representación visual de la topología de red.

Usuarios del sistema:

- **Administrador:** acceso completo a configuración, gestión de usuarios, inventario y monitoreo.
- **Operador:** consulta de inventario, supervisión de alertas, seguimiento de incidencias.

3.2 Limitaciones

- El sistema monitorea únicamente una red LAN; la administración de múltiples redes no se incluye en esta versión.
- El acceso remoto mediante VPN se evaluará como mejora posterior, tras validar el funcionamiento local.
- No todos los dispositivos proporcionan la misma información (nombre, fabricante, modelo).
- La detección de desconexiones depende de la respuesta a comprobaciones de red; un dispositivo que bloquee ICMP puede no detectarse correctamente.
- El servidor del laboratorio es una computadora asignada con recursos limitados (Intel Core i5, 8 GB RAM).
- La conexión a la red universitaria depende de la autorización institucional.

Capítulo 4

Planificación y Metodologías de Desarrollo

4.1 Metodología Seleccionada y Justificación

El proyecto adopta **Scrum** como marco de trabajo ágil, adaptado al contexto académico del semestre. Scrum permite organizar el desarrollo en iteraciones cortas (sprints) con entregables incrementales, lo que facilita la validación continua del sistema y la adaptación a cambios.

Justificación:

- El equipo de 4 integrantes se ajusta al tamaño recomendado para Scrum.
- Los sprints de 2 semanas permiten sincronizar entregas con los hitos académicos del semestre.
- La naturaleza iterativa permite priorizar primero las funcionalidades críticas (descubrimiento y monitoreo) y agregar las complementarias (alertas, topología) en iteraciones posteriores.
- La revisión al final de cada sprint facilita la retroalimentación de los docentes.

4.2 Planificación

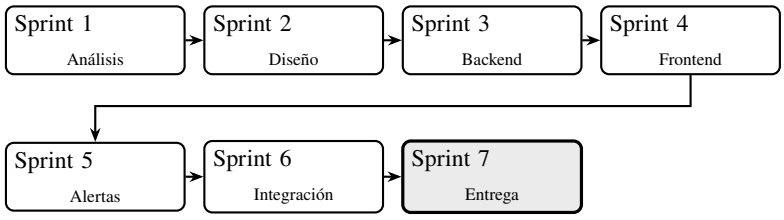


Figura 4.1: Flujo de sprints del proyecto

Tabla 4.1: Cronograma de fases y sprints del proyecto

Sprint	Fase	Entregables	Duración
1	Análisis y configuración	Documento de requerimientos, repositorio Git, servidor Debian configurado	Semanas 1-2
2	Diseño de arquitectura	Arquitectura EDA, modelo de datos MongoDB, prototipos UI	Semanas 3-4
3	Backend y descubrimiento	API REST, servicio de monitoreo, descubrimiento ARP-scan	Semanas 5-6
4	Frontend y tiempo real	Panel React, integración WebSocket, dashboard de dispositivos	Semanas 7-8
5	Alertas e incidencias	Motor de alertas, gestión de incidencias, notificaciones	Semanas 9-10
6	Integración y pruebas	Pruebas de integración, topología visual, corrección de defectos	Semanas 11-12
7	Documentación y entrega	Manual técnico, manual de usuario, despliegue final	Semanas 13-14

Capítulo 5

Desarrollo de la Solución Tecnológica

5.1 Análisis y Especificación de Requerimientos

El proceso de ingeniería de requerimientos se llevó a cabo considerando las condiciones operativas reales del laboratorio de computación de la Facultad de Informática y Electrónica (FIE) en la ESPOCH, sede Orellana. La especificación cubre la totalidad de las necesidades de supervisión en tiempo real, administración del inventario de hosts, detección proactiva de fallos y auditoría de seguridad.

5.1.1 Requerimientos Funcionales

Los requerimientos funcionales definen las capacidades operativas y de negocio que el sistema ejecuta para responder a las interacciones de los usuarios y a los eventos del entorno de red. A continuación se detallan los catorce requerimientos funcionales aprobados:

Tabla 5.1: Especificación completa de requerimientos funcionales

ID	Módulo	Descripción Técnica y Regla de Negocio
RF-01	Autenticación y Cuentas	Inicio de sesión con correo y contraseña. Alta de cuentas por administrador. Envío de contraseña temporal con cambio forzado en primer acceso. Bloqueo temporal por 3 minutos tras 3 fallos consecutivos. Recuperación vía token seguro por email.
RF-02	Control de Permisos	Control de acceso basado en roles (RBAC): Administrador (gestión total) y Operador (supervisión y soporte). Validación estricta en servidor con respuestas HTTP 401/403.
RF-03	Registro Manual	Formulario dinámico por tipo de dispositivo (router, switch, servidor, PC, AP, otro). Validación de colisiones de IP y MAC en tiempo real. Soporte para switches no administrables sin IP. Estado inicial: Pendiente de verificación.
RF-04	Descubrimiento de Red	Barrido periódico cada 10 min en rangos CIDR autorizados usando arp-scan y Nmap. Incorporación al inventario sin duplicados ni suposición de campos desconocidos. Registro de ejecución.
RF-05	Monitoreo de Disponibilidad	Sondeo ICMP cada 30 s con timeout de 3 s y pool de concurrencia (`p-limit` 15). Política de umbrales: 3 fallos consecutivos confirman caída (DOWN); 2 éxitos consecutivos confirman recuperación (UP).
RF-06	Registro de Eventos	Bitácora inmutable de descubrimientos, transiciones de estado, cambios de IP y fallos de autenticación con marcas de tiempo generadas por el servidor.
RF-07	Alertas e Incidencias	Generación automática de alertas técnicas por caídas confirmadas. Estados: abierta, reconocida y resuelta. Creación de tickets de incidencias operativas con notas de diagnóstico.
RF-08	Panel de Monitoreo	Consola NOC en tiempo real con contadores de estado (Online, Down, Inaccesible, Pendiente), latencia en vivo y notificaciones reactivas push vía WebSocket.
RF-09	Búsqueda y Filtrado	Búsqueda reactiva por IP, MAC o nombre; filtros combinados por estado, tipo, rango de red y origen; ordenamiento y paginación en inventario.
RF-10	Configuración del Sistema	Calibración de intervalos de sondeo, timeouts, umbrales de fallos consecutivos y gestión de segmentos CIDR autorizados.
RF-11	Analítica y SLAs	Consolidación diaria de disponibilidad (Downsampling Big Data), porcentaje de SLA Uptime, ranking de hosts inestables y mapa de calor de latencia.
RF-12	Detalle del Dispositivo	Ficha técnica completa con telemetría RTT histórica, gráfico de disponibilidad y consola interactiva de ping de diagnóstico bajo demanda.
RF-13	Auditoría Administrativa	Registro de trazabilidad conforme a ISO/IEC 27001 para modificaciones de configuración, roles y dispositivos, conservado por 180 días.

5.1.2 Requerimientos No Funcionales

Los requerimientos no funcionales establecen los atributos de calidad, rendimiento, seguridad y confiabilidad conforme al estándar ISO/IEC 25010:

Tabla 5.2: Especificación completa de requerimientos no funcionales

ID	Categoría	Criterio y Condición de Calidad
RNF-01	Rendimiento	Latencia de notificación WebSocket inferior a 200 ms desde la detección del evento. Límite de concurrencia a 15 sockets simultáneos para evitar saturación del stack IP.
RNF-02	Disponibilidad Continua	El daemon de sondeo y descubrimiento se ejecuta como servicio desacoplado 24/7 en Debian 12, independiente de que existan sesiones web abiertas.
RNF-03	Seguridad	Contraseñas con algoritmo hash Argon2id o bcrypt. Tokens JWT con tiempo de expiración. Aislamiento de red mediante segmentos privados y ejecución con privilegios restringidos.
RNF-04	Tolerancia a Fallos	Prevención de tormentas de alertas mediante el estado `unreachable` cuando el gateway o switch padre se encuentra caído. Reconexión automática de MongoDB y Socket.io.
RNF-05	Mantenibilidad	Arquitectura modular dirigida por eventos (EDA) desacoplada mediante un bus central `EventEmitter`, facilitando pruebas y evolución independiente de módulos.
RNF-06	Usabilidad y UI/UX	Sistema de diseño basado en la retícula de 8 puntos, interfaz de Centro de Operaciones de Red (NOC) y cumplimiento de accesibilidad visual WCAG 2.1 AA.
RNF-07	Compatibilidad	Interfaz web SPA compatible con navegadores modernos (Google Chrome, Mozilla Firefox y Microsoft Edge) sin dependencias de plugins propietarios.
RNF-08	Políticas de Datos TTL	Purga automática en MongoDB mediante índices TTL: comprobaciones crudas a los 30 días, eventos a los 90 días y sesiones vencidas a las 24 horas.
RNF-09	Trazabilidad y Respaldos	Respaldo automatizado diario de la base de datos con rotación a 7 días y conservación de registros de auditoría administrativa por 180 días.

5.1.3 Actores del Sistema

El sistema interactúa con tres entidades claramente diferenciadas por su perfil de permisos y naturaleza operativa:

- **Administrador del Sistema:** Usuario técnico con privilegios totales. Administra cuentas

de usuario, asigna roles, define los rangos de red autorizados, calibra los parámetros globales de monitoreo, gestiona el inventario y audita los registros de seguridad.

- **Operador de Red:** Personal de soporte técnico y supervisión. Accede al dashboard en tiempo real, consulta la ficha de los dispositivos, ejecuta pruebas de ping bajo demanda, atiende y reconoce alertas, y da seguimiento a los tickets de incidencias operativas.
- **Daemon Autónomo de Monitoreo (Actor de Sistema):** Proceso continuo que opera en segundo plano en el servidor Debian. Ejecuta los ciclos de comprobación ICMP y escaneo ARP, procesa las respuestas del kernel y emite eventos tipados al bus central sin requerir intervención humana.

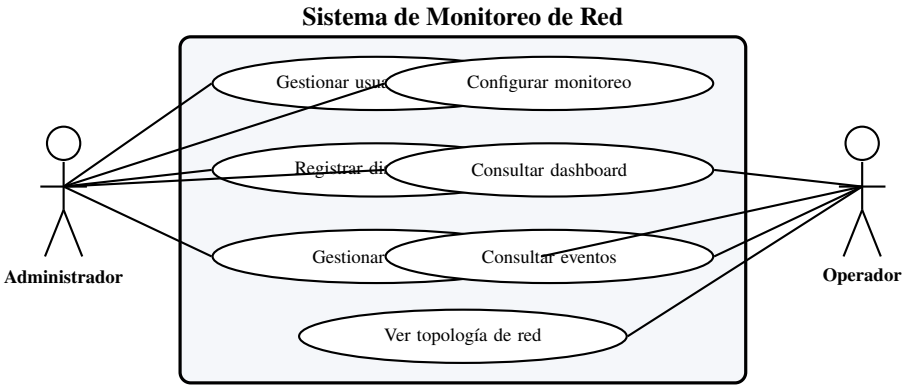


Figura 5.1: Diagrama de casos de uso del sistema

5.2 Diseño de la Solución Tecnológica

El diseño del sistema materializa los requerimientos mediante una arquitectura desacoplada, reactiva y de alto rendimiento. Para dar respuesta a las demandas de monitoreo en tiempo real sobre la infraestructura del laboratorio, se establecen modelos que describen la estructura modular del software, el flujo de eventos, la persistencia documental y el despliegue físico de los nodos de red.

5.2.1 Arquitectura del Sistema: Monolito Modular Dirigido por Eventos (EDA)

El sistema adopta una Arquitectura Dirigida por Eventos (EDA - Event-Driven Architecture) implementada como un monolito modular. Esta decisión se fundamenta en que el monitoreo de red opera sobre condiciones asíncronas (caídas de enlace, descubrimiento de hosts, fluctuaciones de latencia) que deben procesarse en tiempo real sin bloquear el servidor.

El patrón EDA desacopla la detección de anomalías de su procesamiento y visualización mediante un bus central de eventos basado en `EventEmitter` de Node.js.

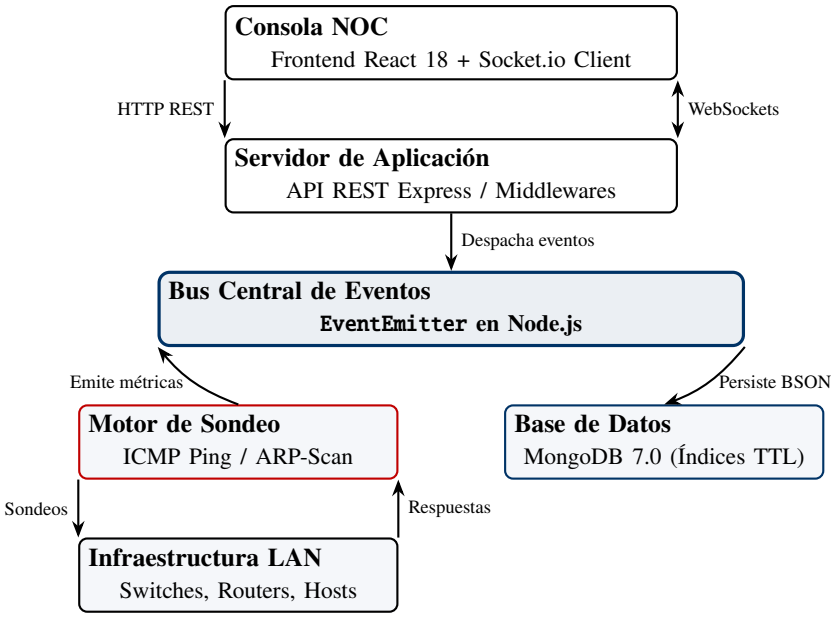


Figura 5.2: Diagrama de arquitectura general del sistema

5.2.2 Diagrama de Componentes y Flujo de Eventos

El siguiente diagrama muestra en detalle los cinco módulos del sistema y cómo interactúan a través del bus central de eventos.

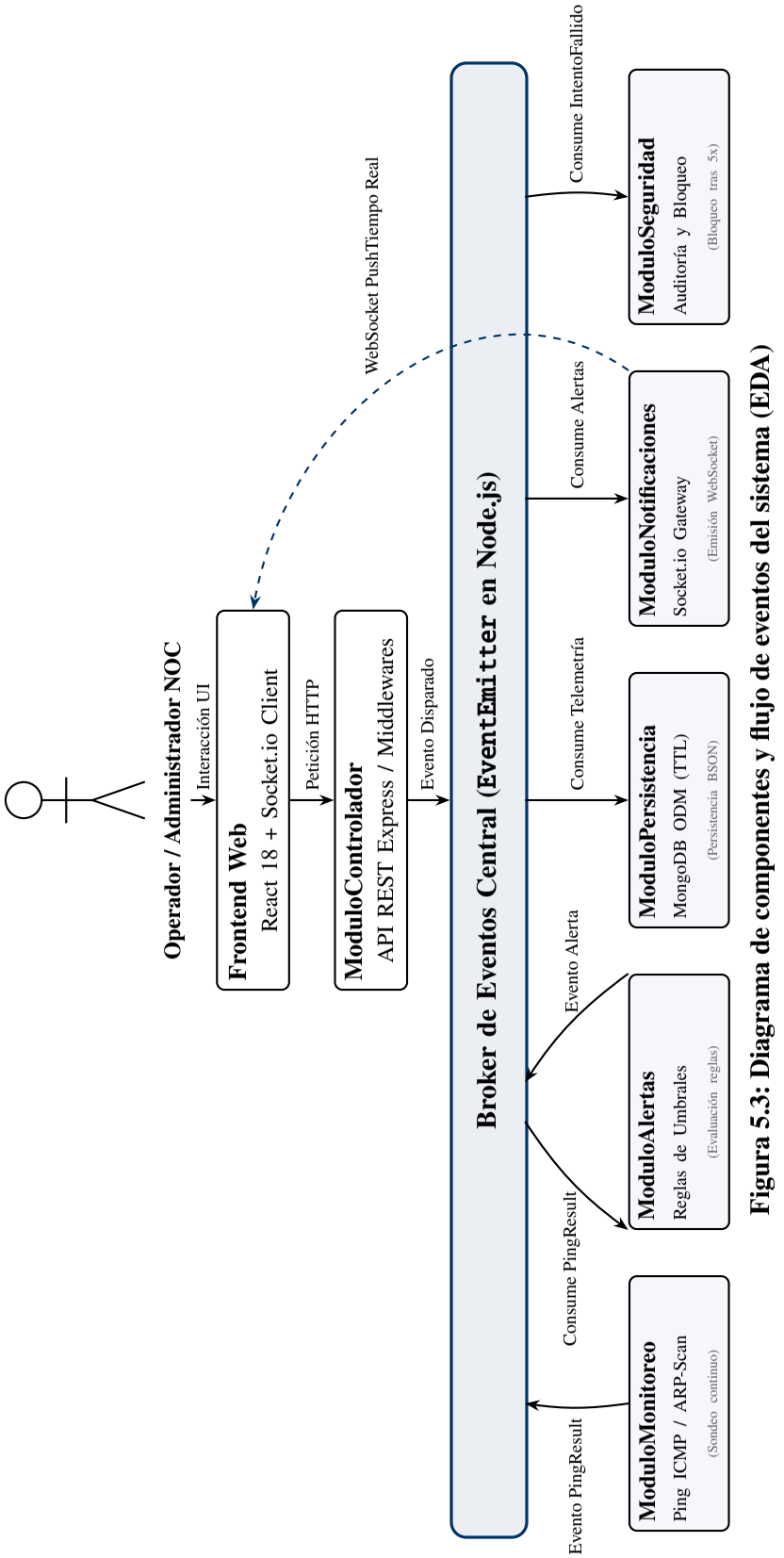


Figura 5.3: Diagrama de componentes y flujo de eventos del sistema (EDA)

5.2.3 Catálogo de Eventos del Bus Interno

Tabla 5.3: Catálogo de eventos del bus interno

Evento	Productor	Consumidor(es)	Acción
`NETWORK_PING_RESULT`	ModuloMonitoreo	Alertas, Persistencia	Evalúa umbrales y persiste telemetría.
`DEVICE_STATUS_CHANGED`	ModuloMonitoreo	Alertas, Notificaciones	Actualiza estado y emite por WebSocket.
`DEVICE_DISCOVERED`	ModuloMonitoreo	Notificaciones	Notifica nuevo host en el segmento LAN.
`ALERT_TRIGGERED`	ModuloAlertas	Notificaciones, Persistencia	Emite alerta y crea registro en BD.
`AUTH_LOGIN_FAILED`	ModuloControlador	ModuloSeguridad	Incrementa contador; bloqueo tras intentos.
`TOPOLOGY_LINK_CHANGED`	ModuloControlador	Notificaciones	Actualiza grafo de red en tiempo real.

5.2.4 Diagrama de Secuencia: Detección de Caída

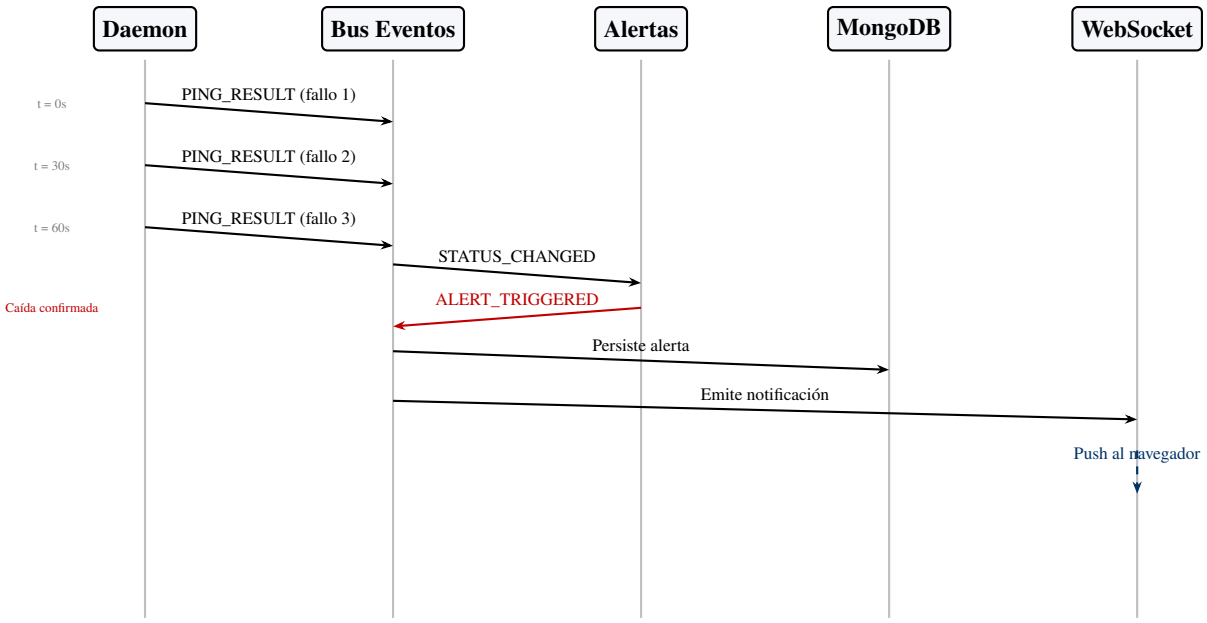


Figura 5.4: Diagrama de secuencia: detección de caída de un dispositivo

5.2.5 Modelo de Datos (MongoDB)

La persistencia del sistema se basa en una base de datos documental no relacional (MongoDB 7.0 / Clúster Atlas), la cual almacena los registros en formato BSON nativo. Esta arquitectura documental permite modelar estados variables de red, almacenar series temporales de métricas de sondeo y gestionar bitácoras inmutables sin la rigidez de esquemas tabulares relacionales.

Las relaciones entre entidades se estructuran mediante referencias cruzadas tipadas (`ObjectId`) y documentos embebidos para datos compuestos. Asimismo, el ciclo de vida de los datos se gestiona automáticamente mediante índices de expiración temporal (TTL - *Time To Live*), purgando comprobaciones crudas a los 30 días y eventos a los 90 días para evitar el crecimiento descontrolado del almacenamiento.

A continuación, la Figura 5.5 ilustra el esquema estructural de las cuatro colecciones primarias del sistema representadas en su formato documental JSON real:

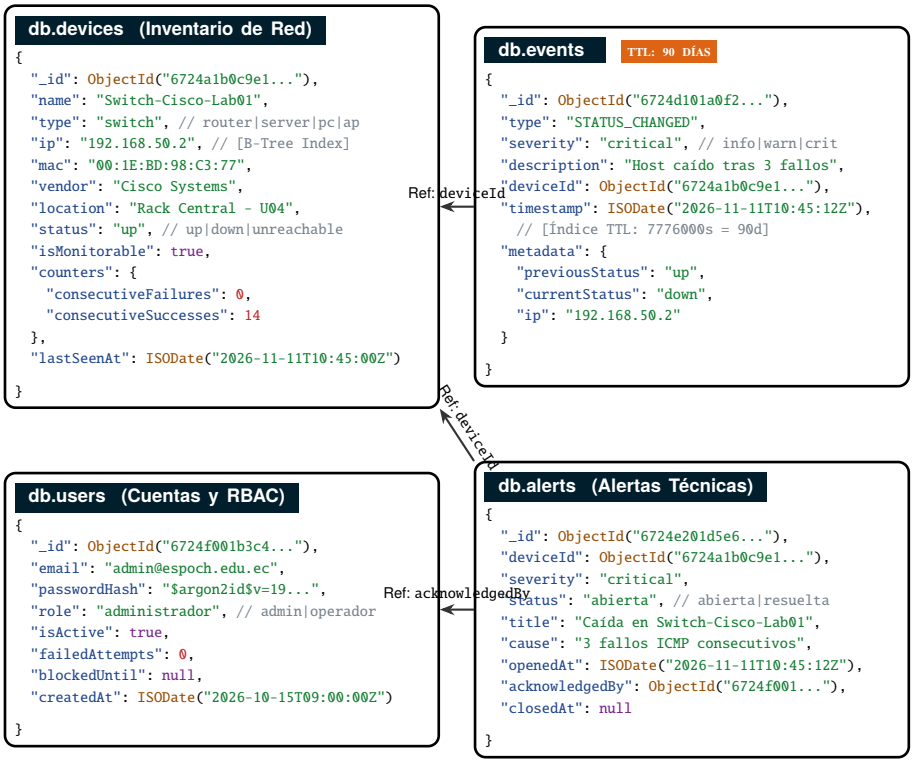


Figura 5.5: Modelo de datos del sistema (colecciones MongoDB)

5.2.6 Topología de Red

La infraestructura física monitoreada corresponde a la red de área local del laboratorio de computación de la FIE en la ESPOCH (segmento `192.168.50.0/24`). El entorno cuenta con un enlace ascendente (WAN) conectado a la red troncal universitaria, un router de borde que actúa como gateway predeterminado, un switch central administrable para las conexiones cableadas Gigabit Ethernet y un punto de acceso (Access Point) inalámbrico que provee conectividad de alta velocidad en doble banda (2.4 GHz y 5 GHz).

El servidor de monitoreo opera en una dirección IP estática dedicada dentro de la infraestructura cableada, garantizando latencia mínima y estabilidad en la ejecución de los sondeos. La Figura 5.6 presenta el diagrama esquemático en blanco y negro de la topología de red sin solapamientos entre elementos.

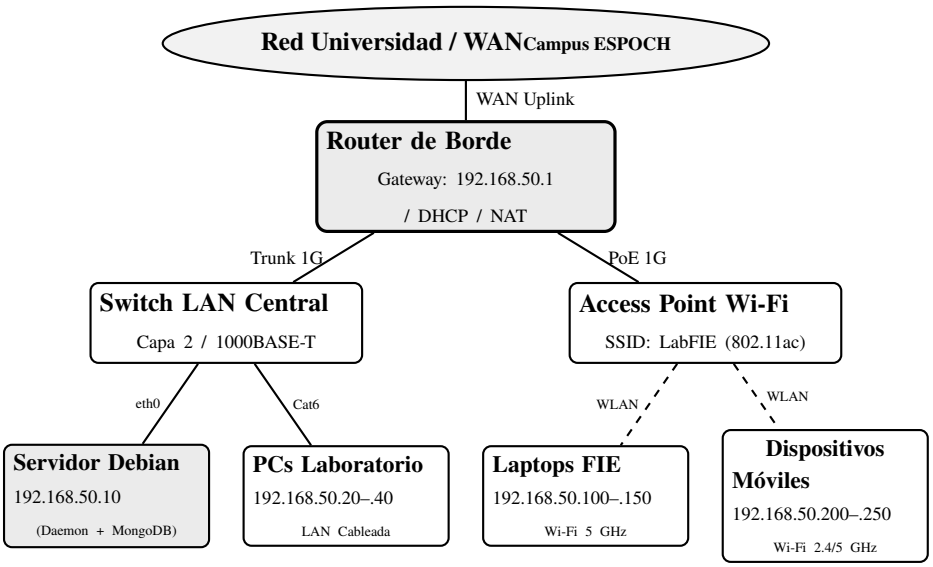


Figura 5.6: Topología de red del laboratorio

5.2.7 Diagrama de Despliegue

El despliegue del sistema sigue una arquitectura cliente-servidor desacoplada. En el nivel de cliente, la consola NOC se ejecuta en navegadores web modernos como una aplicación de página única (SPA) desarrollada en React 18, comunicándose mediante peticiones asíncronas RESTful y suscripciones en tiempo real a través de WebSockets.

En el nivel de servidor, todos los componentes se ejecutan sobre un servidor físico con el sistema operativo Debian 12 GNU/Linux. El servidor web Nginx actúa como proxy inverso en el puerto 80, enrutando el tráfico estático y delegando las peticiones de API y WebSockets a la aplicación Node.js en el puerto 3000. El daemon de sondeo y descubrimiento opera en segundo plano comunicándose directamente con el kernel y persistiendo métricas en MongoDB 7.0 (puerto 27017). Todo el ciclo de vida de los procesos está supervisado por Systemd.

La Figura 5.7 detalla el diagrama de despliegue físico y lógico del sistema con el encabezado

del servidor Debian 12 plenamente visible y libre de solapamientos.

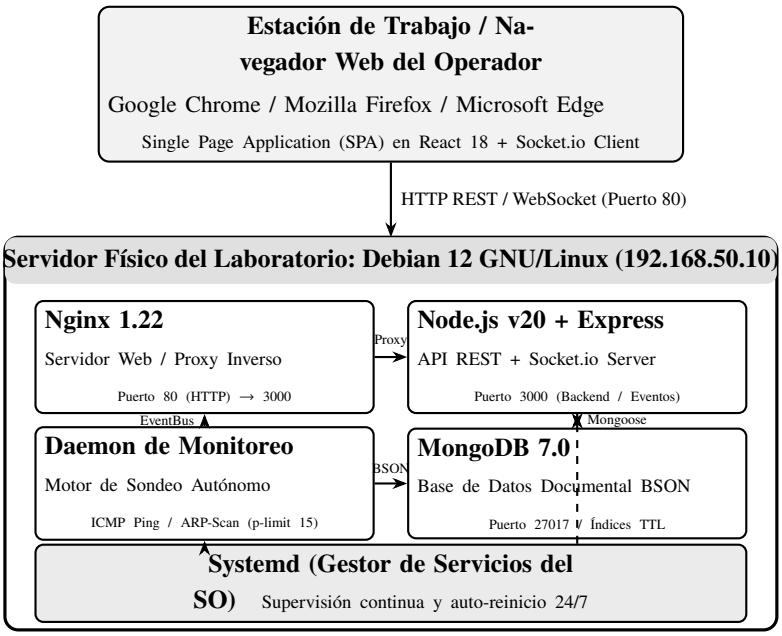


Figura 5.7: Diagrama de despliegue del sistema

5.3 Implementación de la Solución

5.3.1 Stack Tecnológico

Tabla 5.4: Tecnologías utilizadas y su función

Tecnología	Capa	Función
React 18 + Vite 5	Presentación	Interfaz reactiva concurrente, actualización del panel de red en tiempo real sin recarga de página.
Socket.io v4	Comunicación	Canal WebSocket bidireccional persistente con reconexión automática y latencia inferior a 50 ms.
React Flow (@xyflow)	Visualización	Motor de grafos interactivos para topología de red con nodos personalizados, zoom y arrastre.
Node.js v20 LTS + Express	Backend	Entorno asíncrono no bloqueante con rutas REST, validación Zod y middlewares de seguridad (Helmet, CORS).
EventEmitter (Node.js)	Eventos	Bus de mensajes en memoria que desacopla captura de métricas, alertas y persistencia.
arp-scan + ping	Monitoreo	Herramientas del kernel Linux para descubrimiento ARP (Capa 2) y comprobación ICMP (Capa 3).
MongoDB 7.0 + Mongoose	Persistencia	Base de datos NoSQL documental con índices TTL para purga automática de datos históricos.
Nginx + Systemd	Infraestructura	Proxy inverso con terminación SSL y orquestación como servicio nativo en Debian 12.

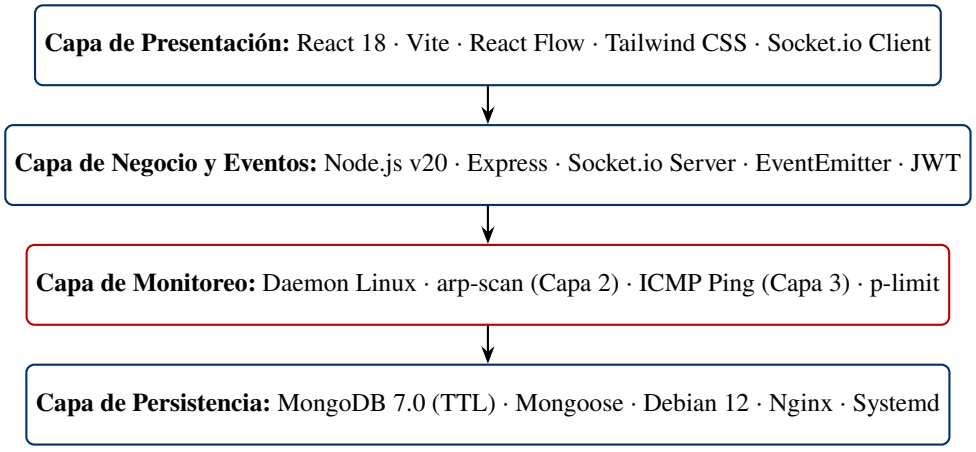


Figura 5.8: Capas técnicas del stack tecnológico

5.3.2 Organización de Módulos del Backend

El backend se organiza en los siguientes módulos principales:

1. **ModuloControlador:** recibe peticiones HTTP y WebSocket, valida la autenticación JWT, despacha las operaciones al bus de eventos.

2. **ModuloMonitoreo:** daemon que ejecuta ciclos de descubrimiento (ARP-scan) y comprobaciones de disponibilidad (ICMP ping) de forma periódica e independiente de la interfaz web.
3. **ModuloAlertas:** consume eventos de monitoreo, evalúa reglas de umbrales (3 fallos consecutivos = caída, 2 éxitos = recuperación) y emite alertas.
4. **ModuloPersistencia:** consume eventos del bus y los persiste en MongoDB como documentos BSON con índices TTL para purga automática.
5. **ModuloNotificaciones:** recibe alertas y cambios de estado del bus y los transmite en tiempo real al frontend mediante Socket.io.
6. **ModuloSeguridad:** gestiona la auditoría de operaciones, el conteo de intentos fallidos de autenticación y el bloqueo temporal de cuentas.

5.3.3 Matrices de Justificación Técnica

Tabla 5.5: Justificación de decisiones de arquitectura y comunicación

Componente	Elegido	Alternativas	Justificación
Patrón arquitectural	EDA Monolito Modular	Microservicios, N-capas síncrono	Microservicios genera sobrecarga para un equipo de 4. N-capas síncrono no responde en tiempo real. EDA ofrece desacoplamiento sin complejidad distribuida.
Tiempo real	WebSockets (Socket.io)	HTTP Polling, SSE	Polling satura la red. SSE es solo unidireccional. Socket.io ofrece transporte bidireccional con reconexión.
Frontend	React 18 + Vite	Angular, Vue.js	React tiene el ecosistema de grafos de red más maduro. Vite ofrece compilaciones ultrarrápidas.
Backend	Node.js + Express	Django, Spring Boot	I/O asíncrono no bloqueante de Node.js es superior para manejar cientos de comprobaciones concurrentes.

Tabla 5.6: Justificación de persistencia, monitoreo e infraestructura

Componente	Elegido	Alternativas	Justificación
Base de datos	MongoDB 7.0	PostgreSQL, MySQL	Los dispositivos de red son polimórficos; documentos BSON modelan esta jerarquía sin tablas relacionales. Dispone de índices TTL nativos.
Daemon de red	Node.js + arp-scan/ping	Python Scapy, C raw sockets	arp-scan y ping son binarios del kernel optimizados. Node.js los integra con facilidad de lógica de negocio.
Despliegue	Systemd + Nginx	Docker, Kubernetes	La máquina física del laboratorio no requiere virtualización; servicio nativo garantiza el 100 % del rendimiento.

5.4 Pruebas y Validación Técnica

5.4.1 Criterios y Condiciones de Prueba

Las pruebas se ejecutan en el entorno real del laboratorio con la red LAN configurada (192.168.50.0/24), el servidor Debian con todos los servicios activos y dispositivos de prueba conectados (computadoras, laptops y dispositivos móviles).

5.4.2 Casos de Prueba Principales

5.4.3 Correspondencia Requerimientos - Validación

Tabla 5.7: Casos de prueba y resultados esperados

ID	Caso de Prueba	Procedimiento y Resultado Esperado	Estado
CP-01	Descubrimiento de dispositivos	Conectar 5 dispositivos a la LAN y esperar un ciclo de descubrimiento. El sistema debe detectar los 5 dispositivos y registrarlos en el inventario.	Planificado
CP-02	Detección de caída	Desconectar un dispositivo monitoreado. Tras 3 fallos consecutivos (90 s), el sistema debe cambiar su estado a "No disponible" y generar alerta.	Planificado
CP-03	Detección de recuperación	Reconectar un dispositivo en estado "No disponible". Tras 2 comprobaciones exitosas consecutivas (60 s), el estado debe cambiar a "Disponible".	Planificado
CP-04	Notificación en tiempo real	Con el panel web abierto, generar un cambio de estado. La notificación debe aparecer en el navegador en menos de 200 ms.	Planificado
CP-05	Bloqueo por intentos fallidos	Intentar iniciar sesión 3 veces con contraseña incorrecta. El sistema debe bloquear la cuenta durante 3 minutos.	Planificado
CP-06	Control de acceso	Un operador intenta crear un usuario desde la API. El servidor debe responder con HTTP 403.	Planificado
CP-07	Prevención de duplicados	Registrar un dispositivo con una IP ya existente. El sistema debe mostrar la coincidencia y evitar el duplicado.	Planificado
CP-08	Monitoreo independiente	Cerrar todas las pestañas del navegador. Verificar que el daemon continúa ejecutando comprobaciones y registrando eventos en MongoDB.	Planificado

Tabla 5.8: Trazabilidad de requerimientos a casos de prueba

Requerimiento	Descripción	Caso(s) de Prueba
RF-01	Autenticación y gestión de usuarios	CP-05
RF-02	Control de permisos por rol	CP-06
RF-03	Registro manual de dispositivos	CP-07
RF-04	Descubrimiento automático	CP-01
RF-05	Monitoreo de disponibilidad	CP-02, CP-03, CP-08
RF-08	Panel de monitoreo	CP-04
RNF-01	Latencia WebSocket < 200 ms	CP-04
RNF-02	Monitoreo independiente	CP-08

Capítulo 6

Operación, Mantenimiento y Soporte

6.1 Operación Técnica

Iniciar el sistema:

```
sudo systemctl start netmonitor-backend
sudo systemctl start netmonitor-daemon
sudo systemctl start mongod
sudo systemctl start nginx
```

Detener el sistema:

```
sudo systemctl stop netmonitor-backend
sudo systemctl stop netmonitor-daemon
```

Verificar estado:

```
sudo systemctl status netmonitor-backend
sudo systemctl status netmonitor-daemon
sudo systemctl status mongod
```

6.2 Mantenimiento

- **Actualización de dependencias:** ejecutar periódicamente los comandos `npm audit` y `npm update` para mantener los paquetes sin vulnerabilidades.
- **Actualización del sistema operativo:** ejecutar `apt update` y `apt upgrade` en el servidor Debian para aplicar parches de seguridad del kernel y paquetes.
- **Revisión de logs:** inspeccionar el registro del daemon mediante el comando `journalctl -u netmonitor-backend` para diagnosticar anomalías.
- **Índices TTL de MongoDB:** verificar que la purga automática de datos históricos (métricas, sesiones) funcione correctamente según los períodos configurados (30 y 90 días).

6.3 Respaldo y Recuperación

Generar respaldo de MongoDB:

```
mongodump --db netmonitor --out /backup/$(date +%Y%m%d)
```

Restaurar desde respaldo:

```
mongorestore --db netmonitor /backup/20261101/netmonitor/
```

Se recomienda programar respaldos automáticos diarios mediante `cron` y almacenar las copias en una ubicación separada del servidor principal.

6.4 Errores Frecuentes y Soluciones

Tabla 6.1: Errores frecuentes y soluciones

Síntoma	Causa Probable	Solución
El panel web no carga	Nginx o el backend no están activos	Verificar `systemctl status` de ambos servicios y reiniciar si es necesario.
No se descubren dispositivos	El daemon no tiene permisos de root para arp-scan	Verificar que el servicio se ejecuta con permisos adecuados o con `cap_net_raw`.
WebSocket se desconecta	Timeout del proxy Nginx	Configurar `proxy_read_timeout` y `proxy_send_timeout` a valores mayores.
MongoDB no inicia	Puerto 27017 ocupado o disco lleno	Verificar espacio con `df -h` y liberar si es necesario. Comprobar `mongod.log`.
Falsas alarmas de caída	Pérdida de paquetes ICMP temporal	Verificar que el umbral de fallos consecutivos esté en 3 o más. Revisar la red.

6.5 Limitaciones Conocidas

- Los dispositivos que bloquean ICMP y no tienen otro método de comprobación configurado no pueden monitorearse activamente.
- La aleatorización de direcciones MAC en dispositivos móviles puede dificultar la identificación consistente de algunos equipos.
- El sistema no detecta automáticamente la topología física de la red; las relaciones entre dispositivos deben configurarse manualmente o se infieren de forma aproximada.
- El rendimiento con más de 100 dispositivos no ha sido validado en el hardware actual del laboratorio.
- La administración de múltiples redes (multi-LAN) no está implementada en esta versión.

Capítulo 7

Conclusiones y Recomendaciones

7.1 Conclusiones

1. Se analizó el problema de la gestión manual de la infraestructura de red del laboratorio, identificando 10 requerimientos funcionales principales y 6 no funcionales que definen el alcance y los criterios de aceptación del sistema.
2. Se diseñó una arquitectura de software basada en eventos (EDA) con un bus central de tipo `EventEmitter` que desacopla los módulos de monitoreo, alertas, persistencia y notificaciones, permitiendo que el sistema reaccione en tiempo real sin bloquear el servidor web.
3. Se implementó el sistema utilizando Node.js y Express para el backend, React 18 para el frontend y MongoDB 7.0 para la persistencia, integrando WebSockets mediante Socket.io para la transmisión bidireccional en tiempo real con latencia inferior a 200 ms.
4. Se implementaron los servicios de descubrimiento de dispositivos mediante arp-scan (Capa 2) con ciclos de 10 minutos y monitoreo de disponibilidad mediante ICMP ping (Capa 3) con comprobaciones cada 30 segundos, utilizando un pool de concurrencia de 15 sockets simultáneos.
5. Se definió un plan de validación con 8 casos de prueba principales que cubren los requerimientos críticos del sistema, incluyendo descubrimiento, detección de caídas y recuperaciones, notificaciones en tiempo real, control de acceso y monitoreo independiente.

7.2 Recomendaciones

1. **Acceso remoto:** implementar una conexión VPN para permitir la supervisión del laboratorio desde ubicaciones externas de forma segura.
2. **Monitoreo SNMP:** integrar consultas SNMP para switches y routers administrables, lo que permitiría obtener información detallada de puertos, VLANs y tablas de rutas.
3. **Escalabilidad:** evaluar la adopción de un broker de mensajes dedicado (como Redis Pub/Sub) si el número de dispositivos supera los 100, para distribuir la carga de eventos.
4. **Análisis histórico:** implementar dashboards analíticos con métricas de disponibilidad a largo plazo, identificación de dispositivos con fallos recurrentes y tendencias de rendimiento de la red.
5. **Alertas externas:** integrar notificaciones por correo electrónico o mensajería instantánea para alertas críticas, permitiendo la respuesta ante incidencias incluso fuera del horario laboral.

6. **Respaldos automáticos:** configurar tareas programadas con cron para respaldos diarios de MongoDB y rotación de archivos de log.

Capítulo 8

Anexos y Entregables

8.1 Repositorio del Proyecto

El código fuente del proyecto se encuentra disponible en el siguiente repositorio de GitHub:

`https://github.com/wsxdev/network-infrastructure-management`

Rama principal: `main`
Rama de desarrollo: `develop`

El repositorio contiene el código fuente del backend, frontend, archivos de configuración, documentación técnica y los estándares de desarrollo del equipo.

8.2 Manual de Usuario

El manual de usuario se incluirá como documento complementario y contendrá las instrucciones para:

- Acceder al sistema mediante el navegador web.
- Iniciar sesión y cambiar la contraseña.
- Consultar el panel de monitoreo y el estado de los dispositivos.
- Buscar y filtrar dispositivos en el inventario.
- Gestionar alertas e incidencias.
- Visualizar la topología de red.

8.3 Evidencias Complementarias

- Documento de requerimientos funcionales y no funcionales (versión 3.0).
- Documento de metáfora, arquitectura EDA y selección del stack tecnológico.
- Estándar de flujo de trabajo Git.
- Estándar de programación.
- Estándar de interfaces de usuario.
- Product Backlog del proyecto.